



EAST WEST UNIVERSITY

Department of Computer Science and Engineering

Lab Report

Course Name: Operating System

Course Code: CSE 325

Section No: 11

Topic: CPU Scheduling

Date of submission: 25.2.2026

Submitted To:

Prof. Dr. Md. Motaharul Islam

Adjunct Professor

Department of Computer Science & Engineering

Submitted By:

Name: Kamrul Hasan

Student ID: 2023-3-60-148

1. First-Come, First-Served (FCFS) Scheduling:

Input:

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int arrivalTime;
```

```
    int burstTime;
```

```
    int waitingTime;
```

```
    int turnaroundTime;
```

```
};
```

```
void sortByArrivalTime(struct Process p[], int n) {
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = i + 1; j < n; j++) {
```

```
            if (p[i].arrivalTime > p[j].arrivalTime) {
```

```
                struct Process temp = p[i];
```

```
                p[i] = p[j];
```

```
                p[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void FCFS_NonPreemptive(struct Process p[], int n) {
```

```

int currentTime = 0;
float totalWaitingTime = 0;

sortByArrivalTime(p, n);

for (int i = 0; i < n; i++) {
    if (currentTime < p[i].arrivalTime) {
        currentTime = p[i].arrivalTime;
    }

    p[i].waitingTime = currentTime - p[i].arrivalTime;
    currentTime += p[i].burstTime;
    p[i].turnaroundTime = p[i].waitingTime + p[i].burstTime;

    totalWaitingTime += p[i].waitingTime;
}

printf("PID\tArrival\tBurst\tWaiting\tTurnaround\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrivalTime, p[i].burstTime,
        p[i].waitingTime, p[i].turnaroundTime);
}

printf("Average Waiting Time = %.2f\n", totalWaitingTime / n);

```

```

}

int main() {
    int n;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for Process %d: ", i + 1);
        p[i].pid = i + 1;
        scanf("%d %d", &p[i].arrivalTime, &p[i].burstTime);
    }

    FCFS_NonPreemptive(p, n);

    return 0;
}

```

Output:

```

Enter number of processes: 3
Enter arrival time and burst time for Process 1: 1 2
Enter arrival time and burst time for Process 2: 2 5
Enter arrival time and burst time for Process 3: 0 3
PID    Arrival Burst    Waiting Turnaround
3       0       3       0       3
1       1       2       2       4
2       2       5       3       8
Average Waiting Time = 1.67

```

2. Shortest Job First (SJF):

Input:

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX_PROCESSES 100
```

```
typedef struct {
```

```
    int pid;
```

```
    int arrival_time;
```

```
    int burst_time;
```

```
    int remaining_time;
```

```
    int waiting_time;
```

```
    int completion_time;
```

```
    int start_time;
```

```
} Process;
```

```
void SJF_NonPreemptive(Process p[], int n) {
```

```
    int time = 0, completed = 0;
```

```
    int is_completed[MAX_PROCESSES] = {0};
```

```
    while (completed != n) {
```

```
        int idx = -1;
```

```
        int min_burst = INT_MAX;
```

```
        for (int i = 0; i < n; i++) {
```

```
            if (p[i].arrival_time <= time && !is_completed[i]) {
```

```

        if (p[i].burst_time < min_burst) {
            min_burst = p[i].burst_time;
            idx = i;
        }
    }
}

```

```

if (idx != -1) {
    p[idx].start_time = time;
    p[idx].waiting_time = time - p[idx].arrival_time;
    time += p[idx].burst_time;
    p[idx].completion_time = time;
    is_completed[idx] = 1;
    completed++;
} else {
    time++;
}
}

```

```

double avg_wait = 0;
printf("Non-Preemptive SJF:\n");
printf("PID\tArrival\tBurst\tWaiting\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
p[i].waiting_time);
    avg_wait += p[i].waiting_time;
}

```

```

    printf("Average Waiting Time = %.2f\n\n", avg_wait / n);
}

void SJF_Preemptive(Process p[], int n) {
    int time = 0, completed = 0, prev = -1;
    int shortest = -1;
    int min_rem_time = INT_MAX;

    for (int i = 0; i < n; i++) {
        p[i].remaining_time = p[i].burst_time;
    }

    while (completed != n) {
        shortest = -1;
        min_rem_time = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (p[i].arrival_time <= time && p[i].remaining_time > 0) {
                if (p[i].remaining_time < min_rem_time) {
                    min_rem_time = p[i].remaining_time;
                    shortest = i;
                }
            }
        }

        if (shortest == -1) {
            time++;

```

```

        continue;
    }

    if (p[shortest].remaining_time == p[shortest].burst_time)
        p[shortest].start_time = time;

    p[shortest].remaining_time--;
    time++;

    if (p[shortest].remaining_time == 0) {
        p[shortest].completion_time = time;
        p[shortest].waiting_time = p[shortest].completion_time -
p[shortest].arrival_time - p[shortest].burst_time;
        completed++;
    }
}

double avg_wait = 0;
printf("Preemptive SJF:\n");
printf("PID\tArrival\tBurst\tWaiting\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
p[i].waiting_time);
    avg_wait += p[i].waiting_time;
}
printf("Average Waiting Time = %.2f\n", avg_wait / n);
}

```



```
int main() {  
    int n;  
    Process p[MAX_PROCESSES];  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    printf("Enter arrival time and burst time for each process:\n");  
    for (int i = 0; i < n; i++) {  
        p[i].pid = i + 1;  
        printf("P%d Arrival: ", i + 1);  
        scanf("%d", &p[i].arrival_time);  
        printf("P%d Burst: ", i + 1);  
        scanf("%d", &p[i].burst_time);  
    }  
  
    Process p_copy[MAX_PROCESSES];  
    for (int i = 0; i < n; i++)  
        p_copy[i] = p[i];  
  
    SJF_NonPreemptive(p, n);  
    SJF_Preemptive(p_copy, n);  
  
    return 0;  
}
```

Output:

```
Enter number of processes: 3
Enter arrival time and burst time for each process:
P1 Arrival: 0
P1 Burst: 2
P2 Arrival: 1
P2 Burst: 2
P3 Arrival: 2
P3 Burst: 4
Non-Preemptive SJF:
PID      Arrival Burst   Waiting
1         0         2       0
2         1         2       1
3         2         4       2
Average Waiting Time = 1.00

Preemptive SJF:
PID      Arrival Burst   Waiting
1         0         2       0
2         1         2       1
3         2         4       2
Average Waiting Time = 1.00
```

3. Round Robin (RR) Scheduling:

Input:

```
#include <stdio.h>

typedef struct {
    int pid;
    int arrival_time;
    int burst_time;
    int remaining_time;
    int waiting_time;
    int finish_time;
} Process;

int main() {
    int n, time_quantum;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
Process p[n];
```

```
printf("Enter arrival time and burst time for each process:\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    p[i].pid = i + 1;
```

```
    printf("Process %d Arrival Time: ", i + 1);
```

```
    scanf("%d", &p[i].arrival_time);
```

```
    printf("Process %d Burst Time: ", i + 1);
```

```
    scanf("%d", &p[i].burst_time);
```

```
    p[i].remaining_time = p[i].burst_time;
```

```
    p[i].waiting_time = 0;
```

```
    p[i].finish_time = 0;
```

```
}
```

```
printf("Enter Time Quantum: ");
```

```
scanf("%d", &time_quantum);
```

```
int current_time = 0, completed = 0;
```

```
int queue[n], front = 0, rear = 0, visited[n];
```

```
for (int i = 0; i < n; i++) visited[i] = 0;
```

```
while (completed < n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (p[i].arrival_time <= current_time && !visited[i] &&  
p[i].remaining_time > 0) {
```

```
        queue[rear++] = i;
        visited[i] = 1;
    }
}
```

```
if (front == rear) {
    current_time++;
    continue;
}
```

```
int idx = queue[front++];

int exec_time = (p[idx].remaining_time < time_quantum) ?
p[idx].remaining_time : time_quantum;
```

```
current_time += exec_time;
p[idx].remaining_time -= exec_time;
```

```
for (int i = 0; i < n; i++) {
    if (p[i].arrival_time <= current_time && !visited[i] &&
p[i].remaining_time > 0) {
        queue[rear++] = i;
        visited[i] = 1;
    }
}
```

```
if (p[idx].remaining_time > 0) {
    queue[rear++] = idx;
```

```

    } else {
        completed++;
        p[idx].finish_time = current_time;
        p[idx].waiting_time = p[idx].finish_time - p[idx].arrival_time -
p[idx].burst_time;
    }
}

int total_waiting_time = 0;
printf("\nProcess\tArrival\tBurst\tWaiting\n");
for (int i = 0; i < n; i++) {
    total_waiting_time += p[i].waiting_time;
    printf("P%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival_time, p[i].burst_time,
p[i].waiting_time);
}

float avg_waiting_time = (float)total_waiting_time / n;
printf("Average Waiting Time: %.2f\n", avg_waiting_time);

return 0;
}

```

Output:

```
Enter number of processes: 3
Enter arrival time and burst time for each process:
Process 1 Arrival Time: 2
Process 1 Burst Time: 3
Process 2 Arrival Time: 1
Process 2 Burst Time: 4
Process 3 Arrival Time: 3
Process 3 Burst Time: 5
Enter Time Quantum: 3

Process Arrival Burst   Waiting
P2      2       3       2
P2      1       4       6
P3      3       5       5
Average Waiting Time: 4.33
```

4. Priority Scheduling

Input:

```
#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>
```

```
#define MAX 100
```

```
typedef struct {
    int id;
    int arrival_time;
    int burst_time;
    int priority;
    int remaining_time;
    int waiting_time;
    int start_time;
    int completion_time;
    bool is_completed;
} Process;
```

```

void inputProcesses(Process p[], int n) {
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter Arrival Time, Burst Time and Priority for Process P%d: ",
p[i].id);
        scanf("%d %d %d", &p[i].arrival_time, &p[i].burst_time, &p[i].priority);
        p[i].remaining_time = p[i].burst_time;
        p[i].is_completed = false;
    }
}

```

```

void nonPreemptivePriority(Process p[], int n) {
    int time = 0, completed = 0;
    float total_waiting = 0;

    while (completed < n) {
        int idx = -1, highest_priority = 9999;

        for (int i = 0; i < n; i++) {
            if (!p[i].is_completed && p[i].arrival_time <= time) {
                if (p[i].priority < highest_priority) {
                    highest_priority = p[i].priority;
                    idx = i;
                } else if (p[i].priority == highest_priority && p[i].arrival_time <
p[idx].arrival_time) {
                    idx = i;
                }
            }
        }
    }
}

```

```
    }  
}
```

```
if (idx != -1) {  
    p[idx].start_time = time;  
    p[idx].waiting_time = p[idx].start_time - p[idx].arrival_time;  
    total_waiting += p[idx].waiting_time;  
    time += p[idx].burst_time;  
    p[idx].completion_time = time;  
    p[idx].is_completed = true;  
    completed++;  
} else {  
    time++;  
}  
}
```

```
printf("\nNon-Preemptive Priority Scheduling:\n");  
printf("PID\tWaiting Time\n");  
for (int i = 0; i < n; i++)  
    printf("P%d\t%d\n", p[i].id, p[i].waiting_time);  
printf("Average Waiting Time: %.2f\n", total_waiting / n);  
}
```

```
void preemptivePriority(Process p[], int n) {  
    int time = 0, completed = 0;  
    float total_waiting = 0;
```



```

while (completed < n) {
    int idx = -1, highest_priority = 9999;

    for (int i = 0; i < n; i++) {
        if (!p[i].is_completed && p[i].arrival_time <= time) {
            if (p[i].priority < highest_priority) {
                highest_priority = p[i].priority;
                idx = i;
            } else if (p[i].priority == highest_priority && p[i].arrival_time <
p[idx].arrival_time) {
                idx = i;
            }
        }
    }

    if (idx != -1) {
        if (p[idx].remaining_time == p[idx].burst_time)
            p[idx].start_time = time;

        p[idx].remaining_time--;
        time++;

        if (p[idx].remaining_time == 0) {
            p[idx].completion_time = time;
            p[idx].waiting_time = p[idx].completion_time - p[idx].arrival_time -
p[idx].burst_time;
            total_waiting += p[idx].waiting_time;
            p[idx].is_completed = true;
        }
    }
}

```

```
        completed++;  
    }  
    } else {  
        time++;  
    }  
}
```

```
printf("\nPreemptive Priority Scheduling:\n");  
printf("PID\tWaiting Time\n");  
for (int i = 0; i < n; i++)  
    printf("P%d\t%d\n", p[i].id, p[i].waiting_time);  
printf("Average Waiting Time: %.2f\n", total_waiting / n);  
}
```

```
int main() {  
    Process processes[MAX];  
    int n;  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    inputProcesses(processes, n);  
  
    Process copy1[MAX], copy2[MAX];  
    for (int i = 0; i < n; i++) {  
        copy1[i] = processes[i];  
        copy2[i] = processes[i];  
    }
```

```
}

nonPreemptivePriority(copy1, n);

preemptivePriority(copy2, n);

return 0;
}
```

Output:

```
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority for Process P1: 1 2 3
Enter Arrival Time, Burst Time and Priority for Process P2: 0 4 2
Enter Arrival Time, Burst Time and Priority for Process P3: 2 3 5

Non-Preemptive Priority Scheduling:
PID      Waiting Time
P1        3
P2        0
P3        4
Average Waiting Time: 2.33

Preemptive Priority Scheduling:
PID      Waiting Time
P1        3
P2        0
P3        4
Average Waiting Time: 2.33
```